

CSCI-4208: Developing Advanced Web Applications

Week 12: Lecture 24

REST API & Microservices
Crash Course

Overview: REST API & Microservices

1. What is a REST API?

- a. The six REST constraints
- b. The marriage of REST & HTTP
 - i. HTTP Methods
 - ii. HTTP Status Codes

2. Common wrinkles & Exceptions

- a. The abuse of GET
- b. Self documenting ...not!
- c. State matters

3. What is Microservice Architecture

- a. Microservices vs. Monolithic systems
- b. Advantages of Microservices
- c. Disadvantages of Microservices
- d. Microservices & REST

1. What is REST API - Definitions

What is a Web API?

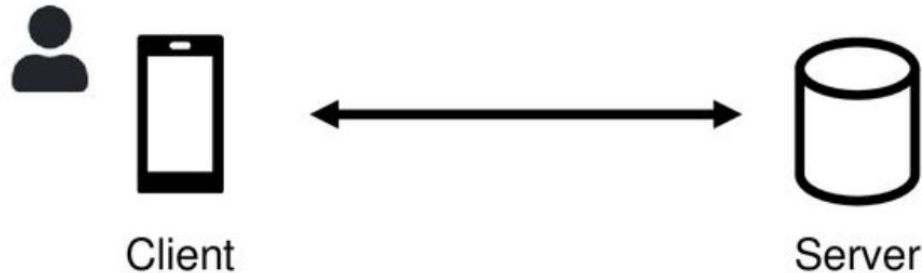
API stands for Application Programming Interface. A web API allows specifically exposed methods of an application to be accessed and manipulated outside of the program itself. Web APIs use web protocols (HTTP, HTTPS, JSON, XML, etc.). For example, a Web API may be used to obtain data from an external resource (such as Playing Cards) without having to actually import the application itself.

What is REST?

REST is short for Representational State Transfer. REST is a software architecture style. A set of rules & conventions for the creation of a Web API. The principles of REST are intended to make APIs & systems that are: efficient, scalable, simple, reliable, & modifiable.

1. What is REST API - Six constraints of REST

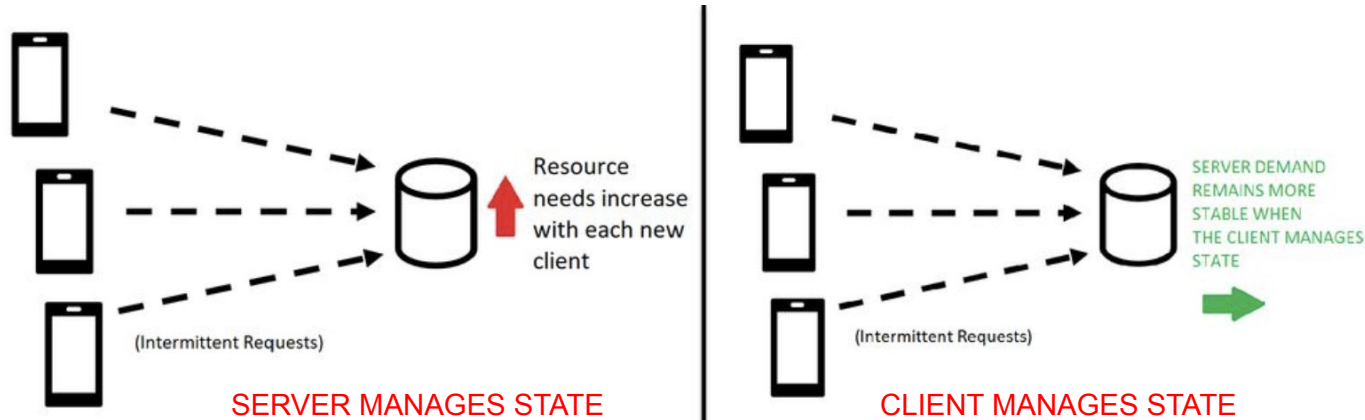
1. **Client-Server architecture** - Simply put, REST systems separate the systems responsible for storing and processing the data (server) from the systems responsible for collecting, requesting, consuming, and presenting the data to a user (the client). This separation should be so distinct that the client and server systems can be improved and updated independently to each other.



An upgrade to the client should not necessitate an upgrade to the server... and vice versa.

1. What is REST API - Six constraints of REST

2. **Statelessness** - As far as the server is concerned, all client requests are treated equally. There's no special, server-side memory of past client activity. The responsibility of managing state (for example, logged in or not) is on the client. This constraint makes the REST approach scalable.



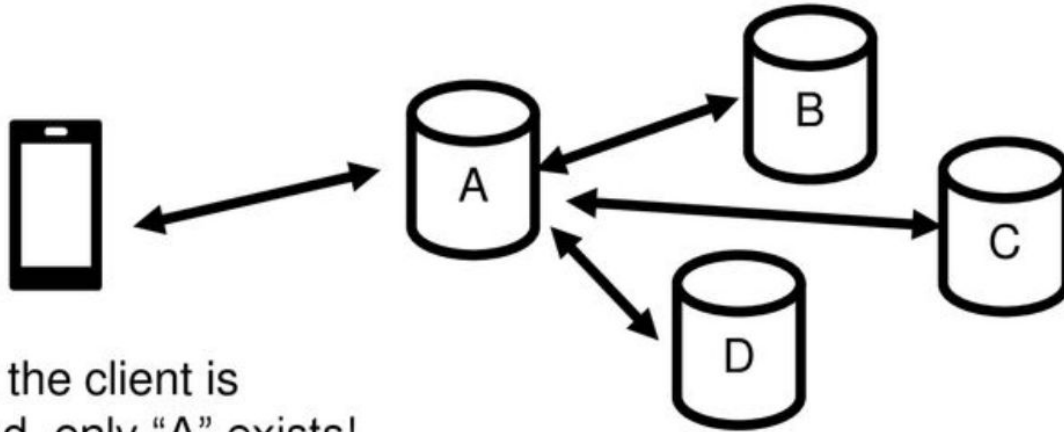
Be advised! In REST each & every resource request is to convey the application state. That means the state gets transferred with each request!

1. What is REST API - Six constraints of REST

3. **Cacheability** - Clients and servers should be able to cache resource data that changes infrequently. For example, there are 50 states in the USA. That's not likely to change soon. So, it is inefficient to build a system that queries a database of states each & every time you need that data. Clients should be able to cache that infrequently updated data & web servers should be able to control the duration of that cache.

1. What is REST API - Six constraints of REST

4. **Layered System** - A client cannot tell whether it is connected directly to an end server, or to an intermediary along the way. Intermediary servers can also improve system scalability.



As far as the client is concerned, only "A" exists!

1. What is REST API - Six constraints of REST

5. **Code of demand (Optional)** - Servers can temporarily extend or customize the functionality of a client by transferring executable code. This constraint is optional.



1. What is REST API - Six constraints of REST

6. **Uniform Interface** - This constraint is comprised of 4 principles. Taken together, these 4 principles require the API to be consistent with HTTP standards & self describing. That means that given an API web address, a server should respond with all the available actions and resources at that address. Each subsequent server response should then contain enough information to take additional actions on the resource.

Four principles:

- Identification of resources (i.e. URI/URL)
- Manipulation of resources through these representations
- Self-descriptive messages
- Hypermedia as the engine of application state

1. What is REST API - The marriage of REST & HTTP

Roy Fielding was one of the principal authors of the HTTP specification. Fielding created the REST constraints with HTTP in mind.

Part of the power of the REST architectural style is that HTTP was already widely in use before the REST constraints were formally defined.

In theory, the six REST constraints we just covered could be applied to other protocols, but in practice REST APIs are HTTP based.

REST  **HTTP**

1. What is REST API - HTTP Methods

GET - the GET method is used to retrieve data from a resource. According to REST conventions, GETs are safe to execute over & over. For example: It is safe to run the following GET request as many times as you want. <http://roll.diceapi.com/json/d6>

POST - The POST method is used to create a data record, or initiate an action. Imagine each POST as being a request to make a amazon purchase. You would want to be careful about doing that more times than necessary!

PUT - The PUT method is used to update an existing data record.

DELETE - The DELETE method is used to delete a record.

1. What is REST API - HTTP Status Codes

Every HTTP request results in a status code to be sent back to the client.

2xx Status codes indicate **success**. The most common status code is 200, OK.

3xx Status codes indicate that the **client needs to do something else** to complete the request. Usually this means making the request from a different location. Status code 301 means the resource has been permanently moved somewhere else.

4xx Status codes indicate that the **client error**. That is, the client sent a request that did not make sense. A classic example of this is '404 Not Found'.

5xx Status codes indicate a **server error**. That means that the request made by the client appears to be fine. But the server is experiencing some difficulty. An example for this is: '500 Internal server error.'

2. Common wrinkles & Exceptions

The REST design constraints are good in theory. However, in practice, no one, not even the largest technology companies, implements them perfectly. Three of the most common constraint violations follow:

- Abuse of the GET method
- APIs are not self documenting
- State Matters

2. Common wrinkles & Exceptions

Abuse of the GET method

GET is assumed to be a safe operation that never modifies the state of the resource. However, it has been often contorted into performing other functions.

Example from 'Deck of Cards API':

A GET endpoint that draws a card from the deck

https://deckofcardsapi.com/api/deck/<<deck_id>>/draw/?count=2

2. Common wrinkles & Exceptions

APIs are not self documenting

This REST ideal is often ignored. Conceptually the principle behind it is the same for defining variables & methods with meaningful names

2. Common wrinkles & Exceptions

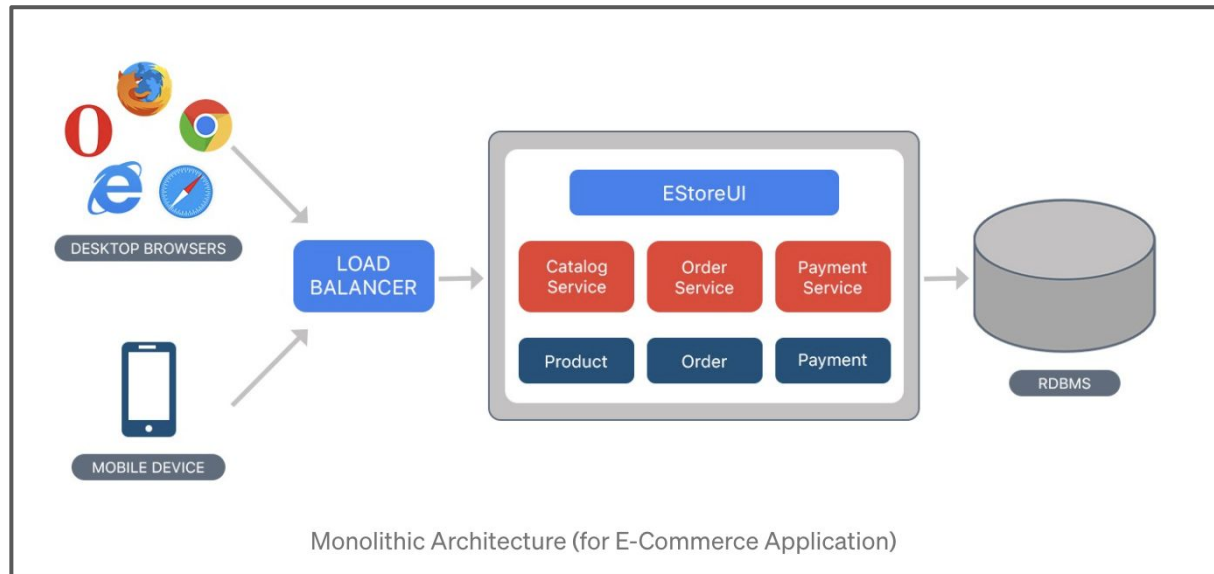
State Matters

In the REST design, the burden of managing state is on the client. That however raises many security concerns. API providers like the efficiency gains made possible by REST, but there is no one consistent approach to managing state.

3. What are Microservices - Monolithic Architecture

What is Monolithic Architecture?

Monolith means composed all in one piece. The Monolithic application describes a single-tiered software application in which different components combined into a single program from a single platform..



3. What are Microservices - Monolithic Architecture

Benefits of Monolithic Architecture?

- Simple to develop — At the beginning of a project it is much easier to go with Monolithic Architecture.
- Simple to test. For example, you can implement end-to-end testing by simply launching the application and testing the UI with Selenium.
- Simple to deploy. You have to copy the packaged application to a server.
- Simple to scale horizontally by running multiple copies behind a load balancer.

3. What are Microservices - Monolithic Architecture

Drawbacks of Monolithic Architecture?

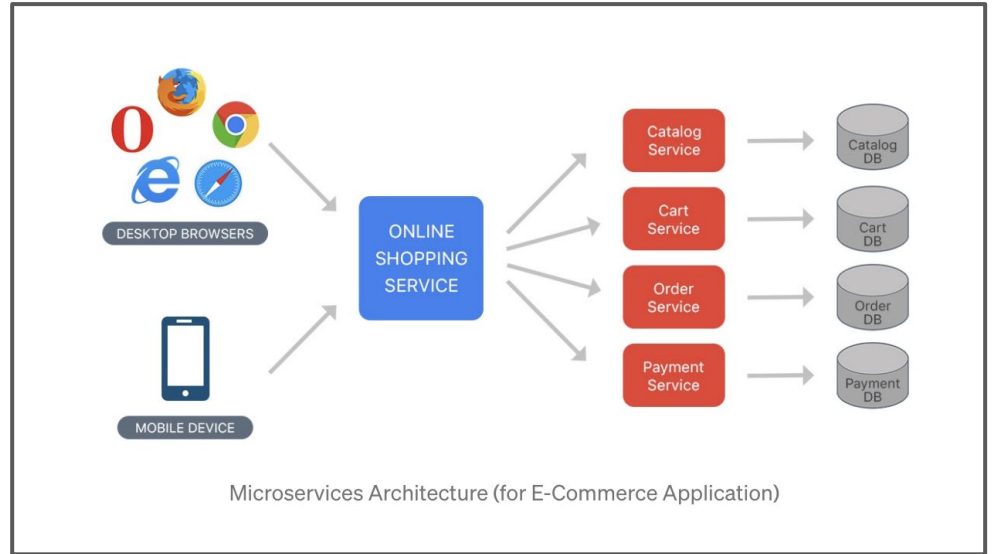
- Maintenance — If Application is too large and complex to understand entirely, it is challenging to make changes fast and correctly.
- The size of the application can slow down the start-up time.
- You must redeploy the entire application on each update.
- Monolithic applications can also be challenging to scale when different modules have conflicting resource requirements.
- Reliability — Bug in any module (e.g. memory leak) can potentially bring down the entire process. Moreover, since all instances of the application are identical, that bug impact the availability of the entire application
- Regardless of how easy the initial stages may seem, Monolithic applications have difficulty to adopting new and advance technologies. Since changes in languages or frameworks affect an entire application, it requires efforts to thoroughly work with the app details, hence it is costly considering both time and efforts.

3. What are Microservices - Microservice Architecture

What is a Microservice Architecture?

Microservices is an architectural style that structures an application as a collection of services that are

- Highly maintainable and testable
- Loosely coupled
- Independently deployable
- Organized around business capabilities



3. What are Microservices - Microservice Architecture

Benefits of Microservice Architecture?

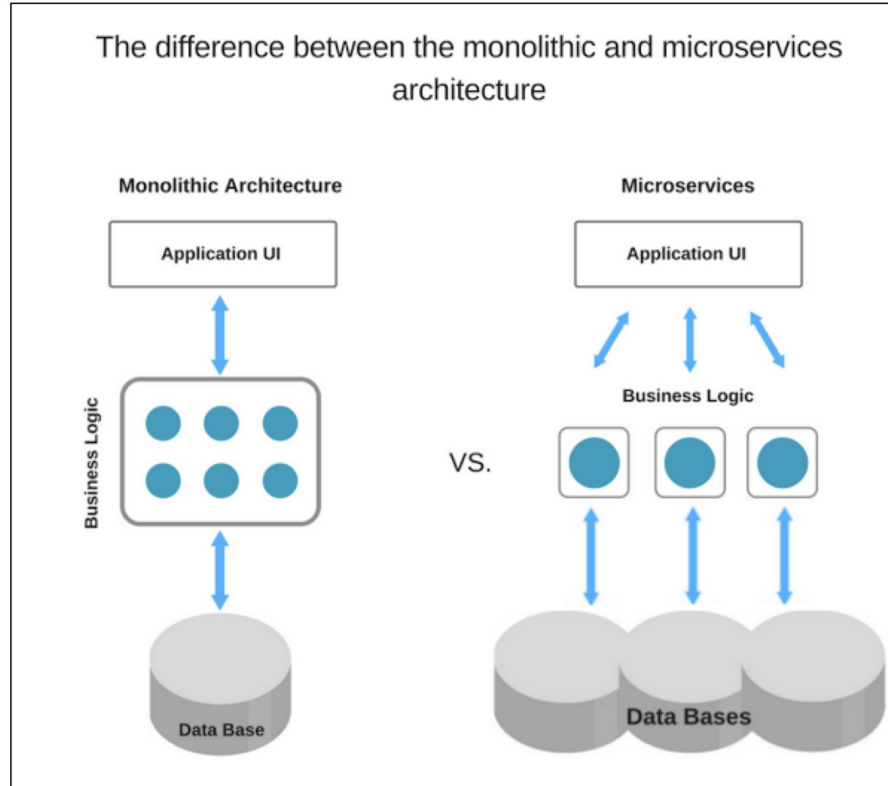
- Microservices Enables the continuous delivery and deployment of large, complex applications.
- Better testability — services are smaller and faster to test.
- Better deployability — services can be deployed independently.
- It enables you to organize the development effort around multiple teams. Each team is responsible for one or more single service. Each team can develop, deploy and scale their services independently of all of the other teams.
- Each microservice is relatively small
- Comfortable for a developer to understand
- The IDE is faster making developers more productive
- The application starts faster, which makes developers more productive, and speeds up deployments
- Improved fault isolation. For example, if there is a memory leak in one service then only that service is affected. The other services continue to handle requests. In comparison, one misbehaving component of a monolithic architecture can bring down the entire system.
- Microservices Eliminates any long-term commitment to a technology stack. When developing a new service you can pick a new technology stack. Similarly, when making major changes to an existing service you can rewrite it using a new technology stack.

3. What are Microservices - Microservice Architecture

Drawbacks of Microservice Architecture?

- Developers must deal with the additional complexity of creating a distributed system.
- Developer tools/IDEs are oriented on building monolithic applications and don't provide explicit support for developing distributed applications.
- Testing is more difficult as compared to Monolith applications.
- Developers must implement the inter-service communication mechanism.
- Implementing use cases that span multiple services without using distributed transactions is difficult.
- Implementing use cases that span multiple services requires careful coordination between the teams.
- Deployment complexity. In production, there is also the operational complexity of deploying and managing a system comprised of many different service types.

3. What are Microservices - Monolithic vs Microservice



Difference b/w Monolith and MicroService Architectures

3. What is Microservices - REST enables Microservices

Microservice Architecture has become increasingly more popular due to the advent of REST APIs. REST APIs have enabled microservices as the REST constraints provides a strategy to easily isolate system components into their own REST API thereby becoming a microservice that may be used by another part of the system.

The End.

COMING SOON... TO A MOODLE NEAR YOU!

Labs

REST API / REST Client Lab

Homework

Design your own REST API